MLOps – HANDS-ON PRACTICAL

Implementing Continuous Delivery (CD) with GitHub Actions

Student Practical Documentation

Item	Details
Course	MLOps
Module	CI/CD and Deployment Automation
Topic	Continuous Delivery (CD)
Practical	Automatic deployment of a static application using GitHub Actions
Platform	Ubuntu / WSL2 + Git + GitHub
Deployment Target	GitHub Pages
Prerequisite	Basic Git, GitHub and the previous CI practical

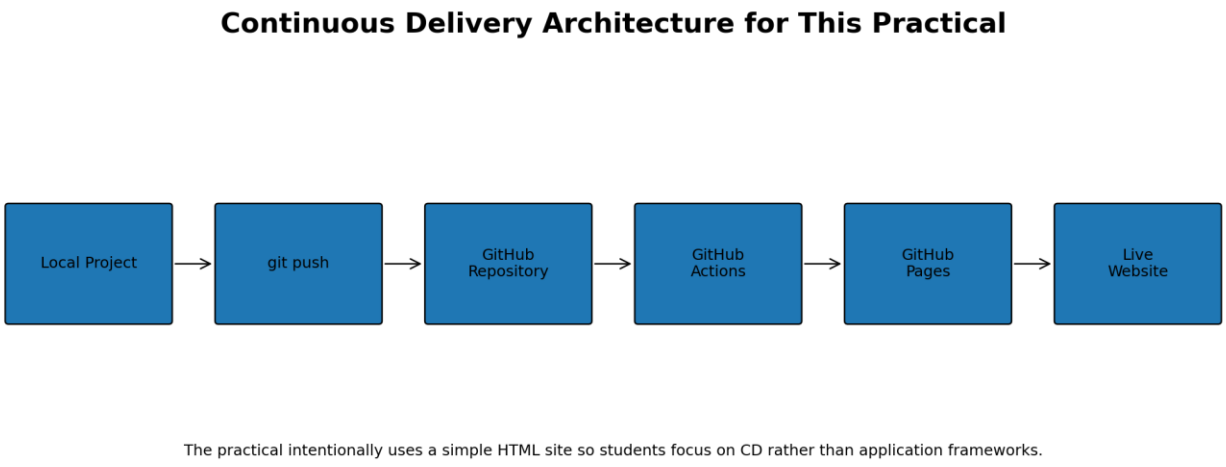


Figure 1. CD architecture used in this practical

1. Learning Objectives

- Understand the purpose of Continuous Delivery (CD).
- Differentiate Continuous Integration, Continuous Delivery and Continuous Deployment.
- Understand how GitHub Actions can automate delivery and deployment.
- Create a simple deployable project and store it in GitHub.
- Configure GitHub Pages as the deployment target.
- Create and understand a GitHub Actions workflow for deployment.
- Trigger a deployment by pushing a change to the main branch.
- Verify a successful deployment and inspect workflow logs.
- Demonstrate Version 1 → Version 2 automatic deployment.
- Observe a deployment failure, identify the failed stage, correct it and rerun the workflow.
- Relate CD to a larger MLOps lifecycle.

2. What is Continuous Delivery?

Continuous Delivery (CD) is the practice of keeping software or an application in a deployable state through an automated and repeatable release process. After code changes are integrated and validated, the delivery process prepares the change for deployment to a target environment.

For this practical, the target is GitHub Pages. The application is deliberately a simple HTML page so that students can concentrate on the CD mechanism rather than on FastAPI, Docker, Kubernetes or model-serving details.

3. CI vs Continuous Delivery vs Continuous Deployment

Aspect	Continuous Integration (CI)	Continuous Delivery (CD)	Continuous Deployment
Main purpose	Integrate and validate changes frequently	Keep validated changes ready for release/deployment	Automatically deploy validated changes
Typical activities	Build, unit tests, validation	Package, publish, prepare release, deployment workflow	Automatic deployment after validation
Human approval	Usually not part of testing	May be required before production deployment	Normally no manual deployment approval
Output	Validated build/change	Deployable release/artifact	Running updated application
This practical	Covered in the previous CI practical	Demonstrated through automated GitHub Pages delivery	The workflow also demonstrates the automatic deployment aspect

Teaching point: CI answers “Is the change validated?” CD answers “Can the validated change be delivered/deployed repeatably?” Continuous Deployment goes one step further by automatically deploying validated changes without a manual deployment step.

4. Why CD is Needed in MLOps

- Manual deployment can be slow and error-prone.
- The same deployment procedure can be repeated consistently.
- A new validated version can be delivered quickly.

- Deployment history can be inspected through workflow runs.
- Teams can connect source-control events to release and deployment automation.
- In MLOps, the same idea can later be applied to model packages, model-serving applications or other ML artifacts.

5. Important CD Concepts

Concept	Meaning
Trigger	An event that starts a workflow, such as a push to main or a manual workflow dispatch.
Workflow	A YAML-defined automation process executed by GitHub Actions.
Job	A collection of related workflow steps executed by a runner.
Runner	The machine/environment on which GitHub Actions executes the job.
Artifact	A packaged output that can be passed to a deployment stage.
Deployment target	The environment or service where the application is delivered.
Environment	A named deployment context, such as github-pages.
Release / deployment	The process of making a new version available in the target environment.
Rollback	Returning to a previously known version when a newer deployment has a problem.

6. CD Architecture

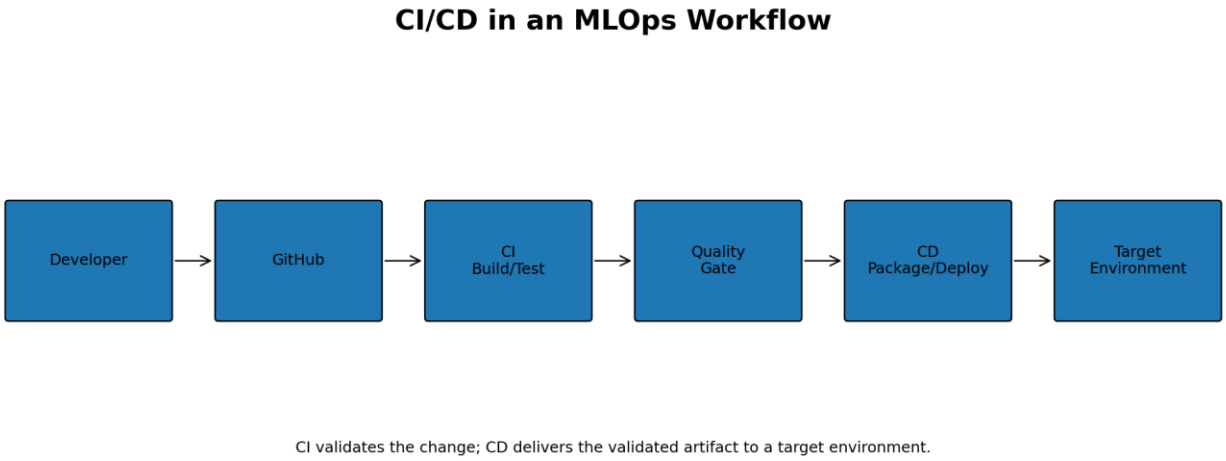


Figure 2. CI/CD architecture in an MLOps workflow

The architecture shows the relationship between validation and delivery. The previous CI practical performs automated testing. Once a change is validated, the CD part can package and deliver it to a target environment.

6.1 Architecture Used in This Hands-on

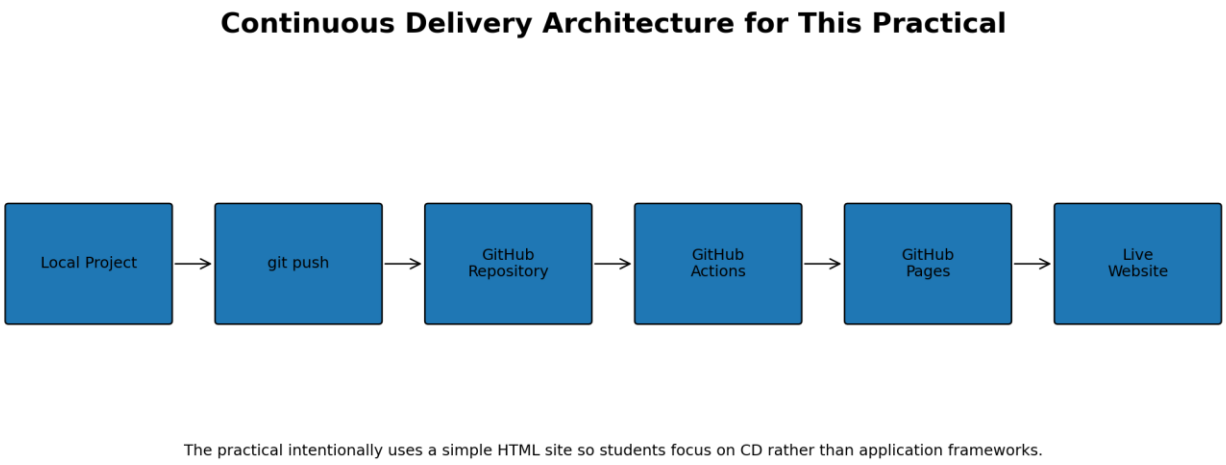


Figure 3. Architecture used for the GitHub Pages CD demonstration

6.2 GitHub Actions Workflow Architecture

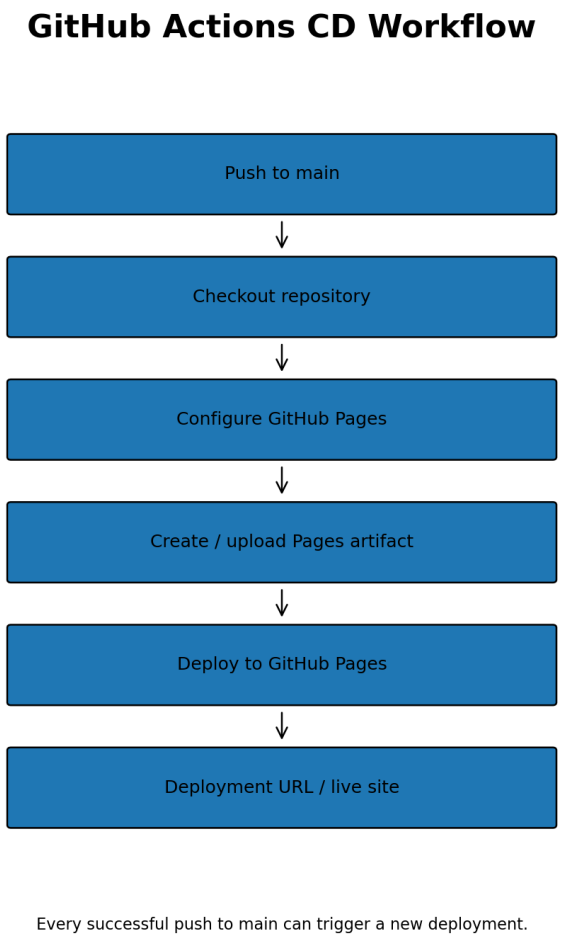


Figure 4. GitHub Actions deployment workflow

The workflow is event-driven. A push to the main branch starts the workflow. The runner checks out the repository, prepares GitHub Pages, creates/uploads the deployment artifact, and performs the deployment.

7. Practical Scenario

We will create a very small static website named cd-demo. The initial site is Version 1. After the first deployment is successful, we will change the page to Version 2. A push to main will trigger GitHub Actions again, and the new version will be deployed automatically.

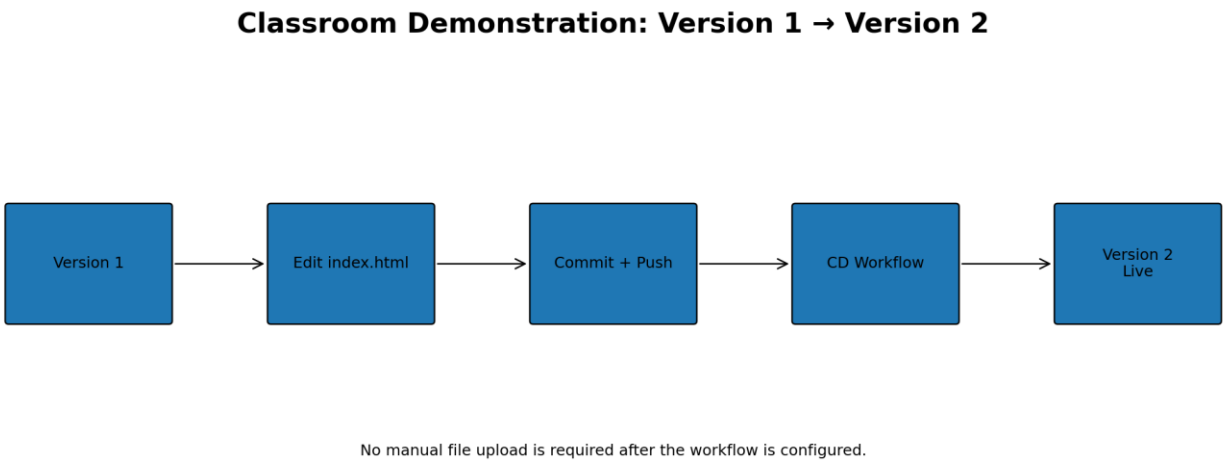


Figure 5. Version 1 → Version 2 classroom demonstration

8. Prerequisites

- Ubuntu / WSL2 environment.
- Git installed and working.
- A GitHub account.
- Permission to create a GitHub repository.
- Basic knowledge of git add, git commit and git push.
- The previous CI practical is recommended because it establishes GitHub Actions fundamentals.

```
git --version
python3 --version
```

9. Hands-on: Create the CD Project

Open Ubuntu / WSL2 and create the project directory:

```
mkdir -p ~/kubeflow-mlops/cd-demo
cd ~/kubeflow-mlops/cd-demo
pwd
```

Expected location will be similar to:

```
/home/<username>/kubeflow-mlops/cd-demo
```

10. Create and Test the Application

Create the HTML file:

```
nano index.html
```

Enter:

```
<!DOCTYPE html>
<html>
<head>
    <title>MLOps CD Demo</title>
</head>

<body>

    <h1>MLOps Continuous Delivery Demo</h1>

    <h2>Version 1</h2>

    <p>
        This application was deployed automatically
        using GitHub Actions.
    </p>

</body>
</html>
```

Save with Ctrl+O, Enter, then Ctrl+X.

For a quick local check, from WSL you can open the file in Windows:

```
explorer.exe index.html
```

The browser should display the MLOps Continuous Delivery Demo and Version 1.

11. Create the Git Repository

```
git init
git branch -M main
git status
git add .
git commit -m "Add initial CD demo"
git status
```

12. Create the GitHub Repository

Create a new GitHub repository. A suitable name is cd-demo. If your classroom repository has another name, use that name consistently in the commands.

```
git remote add origin https://github.com/<your-username>/cd-demo.git
git push -u origin main
```

Never place a GitHub Personal Access Token in source code, workflow files or classroom documents.

Create the CD Workflow

Create the workflow directory:

```
mkdir -p .github/workflows

Create: nano .github/workflows/cd.yml

name: Continuous Delivery

on:
  push:
    branches:
      - main

permissions:
  contents: read
  pages: write
  id-token: write

jobs:

  deploy:

    runs-on: ubuntu-latest

    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}

    steps:

      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Pages
        uses: actions/configure-pages@v5

      - name: Upload website
        uses: actions/upload-pages-artifact@v3
        with:
          path: .

      - name: Deploy to GitHub Pages
        id: deployment
        uses: actions/deploy-pages@v4
```

13. Configure GitHub Pages

Open the repository in GitHub and navigate to:

```
Repository
   ↓
Settings
   ↓
Pages
   ↓
Build and deployment
   ↓
Source
```

```
   ↓
GitHub Actions
```

Do not use the account-level Settings → Pages page. The Pages configuration required here is inside the repository Settings.

GitHub may show a suggested workflow named Static HTML. Select Configure for the Static HTML workflow. This is the route used in the classroom demonstration.

14. Create the GitHub Actions CD Workflow

GitHub's Static HTML Pages workflow can be used as the deployment workflow. The following is a representative workflow structure for this practical. If GitHub generates a newer version of the workflow, keep the generated Pages actions when they provide the same checkout, artifact and deployment stages.

```
name: Deploy static content to Pages

on:
  push:
    branches: ["main"]
  workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: "pages"
  cancel-in-progress: false

jobs:
  deploy:
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}

    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Pages
        uses: actions/configure-pages@v5

      - name: Upload artifact
        uses: actions/upload-pages-artifact@v3
        with:
          path: .

      - name: Deploy to GitHub Pages
        id: deployment
        uses: actions/deploy-pages@v4
```

14.1 Meaning of the Workflow

Workflow element	Purpose
push → main	Automatically starts the workflow when a change is pushed to main.
workflow_dispatch	Allows the workflow to be started manually from GitHub Actions.
permissions	Allows the workflow to read repository contents and deploy to Pages.
runs-on	Specifies the GitHub-hosted runner.
checkout	Gets the repository files into the runner.
configure-pages	Prepares the GitHub Pages deployment.
upload-pages-artifact	Packages the files that will be published.
deploy-pages	Deploys the Pages artifact.
github-pages environment	Represents the deployment environment used by GitHub Pages.

15. Commit and Run the CD Workflow

If you configure the workflow through GitHub's Static HTML → Configure option, GitHub opens the YAML editor. Review the generated workflow and click Commit changes.

```
Suggested commit message:
Add CD workflow for GitHub Pages
```

Because the workflow listens for a push to main, the commit itself can trigger the deployment.

15.1 View the Workflow Run

Open:

```
Repository
   ↓
Actions
   ↓
Deploy static content to Pages
```

Open the latest run. A successful deployment should show stages similar to:

- Set up job
- Checkout
- Setup Pages
- Upload artifact
- Deploy to GitHub Pages
- Post Checkout
- Complete job

16. Verify the Deployment

After a successful run, GitHub Pages provides the site URL. Open the deployment URL and confirm that Version 1 is visible.

Expected result:

```
MLOps Continuous Delivery Demo

Version 1
```

```
This application was deployed automatically
using GitHub Actions.
```
This completes the first deployment. At this point students have seen code move from the repository through GitHub Actions to a live deployment target.

17. Version 1 → Version 2 Automatic Deployment

This is the most important classroom demonstration because it makes the meaning of continuous delivery visible.

Open index.html in GitHub and edit:

```
<h2>Version 1</h2>
```
Change it to:

```
<h2>Version 2</h2>
```
Also change the paragraph to:

```
<p>
    Version 2 was automatically deployed using GitHub Actions.
</p>
```
Commit the change to main:

```
git add index.html
git commit -m "Update application to Version 2"
git push
```
If the edit was performed directly in GitHub, the commit itself is sufficient; a separate local git push is not needed.

17.1 Observe the Automatic Deployment

1. Open the repository Actions page.
2. Open Deploy static content to Pages.
3. Open the new workflow run created by Update index.html.
4. Verify that the deploy job is successful.
5. Open the GitHub Pages site.
6. Refresh the page and confirm that Version 2 is displayed.

Expected workflow:

```
Update index.html
       ↓
push to main
       ↓
GitHub Actions
       ↓
Checkout
       ↓
Setup Pages
       ↓
Upload artifact
       ↓
Deploy to GitHub Pages
       ↓
Version 2 Live
```

18. Demonstrating CD Failure and Recovery

A failure demonstration helps students understand that CD is an automated process with observable stages. The safest classroom approach is to make a temporary workflow mistake and then restore it.

For example, temporarily change the artifact path from:

```
path: .
```

to an invalid path such as:

```
path: wrong-folder
```

Commit and push the workflow change. The deployment should fail because the expected artifact content is not available at the specified location.

Students should observe:

```
🔴 CD FAILED
       ↓
Open failed job
       ↓
Identify failed step
       ↓
Correct workflow
       ↓
Commit + push
       ↓
🟢 CD PASSED
```

Restore the correct artifact path and push again. The purpose is not to create a complicated error, but to demonstrate failure → diagnosis → correction → successful deployment.

19. Complete CD Workflow

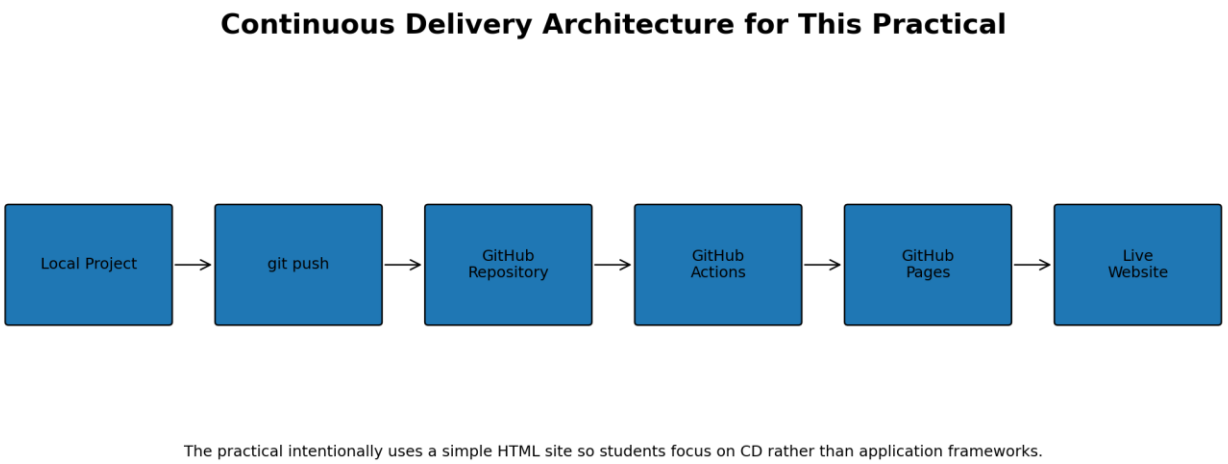


Figure 6. Complete practical workflow

The complete practical can be summarized as: create application → commit to GitHub → configure GitHub Pages → create the Actions workflow → deploy Version 1 → modify to Version 2 → push → automatically deploy Version 2 → inspect the workflow result.

20. CD in an MLOps Lifecycle

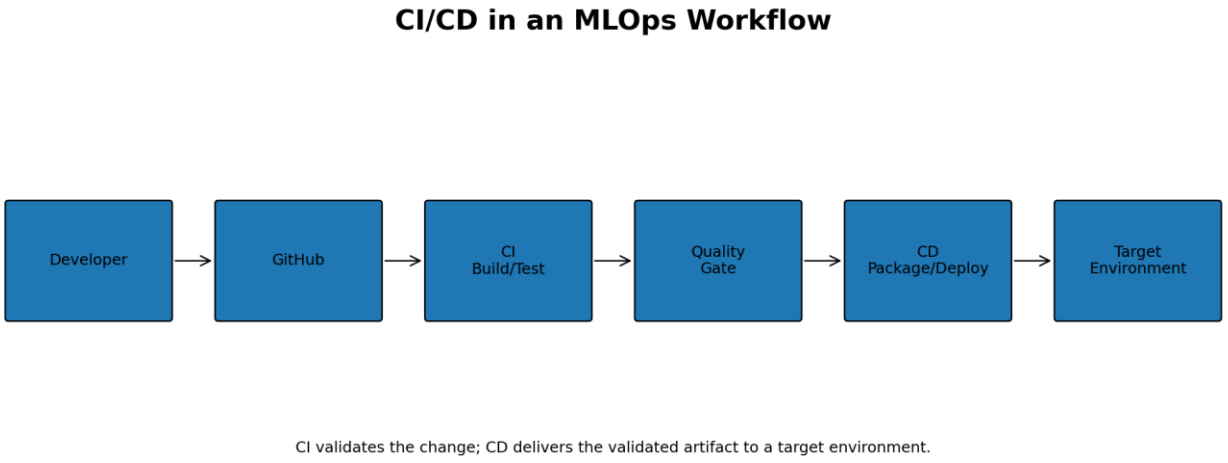


Figure 7. Relationship of CI/CD to the MLOps lifecycle

In a real MLOps system, the deployment target may be an ML application, model-serving system, container platform or another production environment. The present practical intentionally avoids those additional technologies. Its purpose is to establish the CD mechanism first.

21. Troubleshooting

Problem	Check / Action
Pages option appears different	Make sure you are inside the repository Settings → Pages, not account-level Settings → Pages.
No deployment workflow	Open Actions and verify that the Pages workflow file exists under .github/workflows.
Workflow does not trigger	Confirm the workflow listens to push on the main branch and that the commit reached main.
Deployment job fails	Open the failed job and inspect the exact step marked with a red X.
Pages source is not GitHub Actions	Repository Settings → Pages → Build and deployment → Source → GitHub Actions.
Site still shows an older version	Wait for the latest deployment to finish and refresh the site. Confirm the latest Actions run is successful.
Wrong repository branch	Check the branch shown in the workflow trigger and the branch receiving the commit.
Two Pages workflows exist	For a classroom repository, keep one clear deployment workflow to avoid confusion.
Workflow permission problem	Check the permissions block and the repository Pages configuration.
Custom domain is shown	A custom domain is not required for this practical; use the default GitHub Pages domain.

23. Viva / Review Questions

1. What is Continuous Delivery?

2. What is the difference between CI and CD?

3. What is Continuous Deployment?

4. What event triggers the CD workflow in this practical?

5. What is GitHub Actions?

6. What is a GitHub Actions runner?

7. Why is actions/checkout used?

8. Why is GitHub Pages used as the deployment target?

9. What is an artifact in this workflow?

10. What does upload-pages-artifact do?

11. What does deploy-pages do?

12. What happens after a push to main?

13. How do you verify that deployment succeeded?

14. What should you inspect when a deployment fails?

15. How did Version 2 become live without manually uploading the file?

16. How is Continuous Delivery different from Continuous Deployment?

17. How can CD be extended to a real MLOps environment?

24. Quick Command Reference

```
# Create project
mkdir -p ~/kubeflow-mlops/cd-demo
cd ~/kubeflow-mlops/cd-demo

# Check Git
git --version

# Initialize repository
git init
git branch -M main

# Check status
git status

# Stage files
git add .
```

```
# Commit
git commit -m "message"

# Add GitHub remote
git remote add origin https://github.com/<username>/cd-demo.git

# Push
git push -u origin main

# Later pushes
git add .
git commit -m "Update application"
git push
```

25. Practical Outcome

After completing this practical, students should be able to explain Continuous Delivery, differentiate CI/CD concepts, create a simple GitHub-based deployment workflow, configure GitHub Pages as a deployment target, inspect GitHub Actions deployment logs, demonstrate Version 1 → Version 2 automatic deployment, and diagnose a basic deployment failure.

The central learning outcome is the observable automation chain:

```
Developer
   ↓
Git commit / push
   ↓
GitHub
   ↓
GitHub Actions
   ↓
Package / artifact
   ↓
Deployment
   ↓
Live application
```